

Rest and Restful

REST (Representational State Transfer) is an architectural style for designing **networked applications**. It relies on **stateless communication** and standard **HTTP(s)** methods (like GET, POST, PUT, DELETE) to manage resources.

A **RESTful API** is an implementation of REST principles. It provides a way for clients to interact with resources over HTTP by sending requests and receiving responses, typically in JSON format.

Key Concepts of REST:

1. **Statelessness**: Each request from a client contains all the information the server needs to process it.
2. **Resources**: Represented as URLs (e.g., `/users`, `/posts`).
3. **HTTP Methods**:
 - `GET`: Retrieve data. (**CRUD**)
 - `POST`: Create data.
 - `PUT`: Update data.
 - `DELETE`: Remove data.
4. **JSON Data**: Common format for request and response bodies.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Mock database
users = [
    {"id": 1, "name": "Alice"},
    {"id": 2, "name": "Bob"}
]

# Retrieve all users (GET)
@app.route('/users', methods=['GET'])
def get_users():
```

```

    return jsonify(users)

# Retrieve a single user by ID (GET)
@app.route('/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    user = next((u for u in users if u["id"] == user_id), None)
    if user:
        return jsonify(user)
    return jsonify({"error": "User not found"}), 404

# Add a new user (POST)
@app.route('/users', methods=['POST'])
def add_user():
    data = request.json
    new_user = {
        "id": len(users) + 1,
        "name": data["name"]
    }
    users.append(new_user)
    return jsonify(new_user), 201

# Update a user (PUT)
@app.route('/users/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    user = next((u for u in users if u["id"] == user_id), None)
    if user:
        data = request.json
        user["name"] = data.get("name", user["name"])
        return jsonify(user)
    return jsonify({"error": "User not found"}), 404

# Delete a user (DELETE)
@app.route('/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    global users
    users = [u for u in users if u["id"] != user_id]
    return jsonify({"message": "User deleted"}), 200

```

```

if __name__ == '__main__':
    app.run(debug=True)

@app.route('/models', methods=['POST'])
def add_model():
    data = request.json
    #Model name
    #Model api key
    new_model = {
        "id": len(users) + 1,
        "name": data["name"]
    }
    users.append(new_user)
    return jsonify(new_user), 201

@app.route('/models/<int:model>', methods=['POST'])
def add_model():
    model = load_model(model_id)
    data = request.json()
    return
    {
        model_id
        inference
    }

```

Resources and Naming Conventions in RESTful APIs

What are Resources?

In REST, a **resource** is any entity that you want to **expose to clients**. Examples of resources include users, orders, products, blog posts, models, database, vectors, etc.

Each resource is identified by a **URI (Uniform Resource Identifier)**, which is typically structured as a path in the API.

Key Principles for Naming Resources

1. Nouns, Not Verbs

Use nouns to represent resources, as they describe the entities you're working with.

Example:

-  `/users` (represents a collection of users)
-  `/getUsers` (uses a verb, which is not RESTful)

2. Plural Form for Collections

Use plural nouns to represent a collection of resources.

Example:

-  `/model` (collection of users) user
-  `/model` (implies a single user, which should be `/models/{id}`)

3. Hierarchical Relationships

Structure URIs to reflect relationships between resources.

Example:

- `/users/1/orders/` (orders belonging to user with ID 1)
- `/products/42/reviews` (reviews for product with ID 42)

4. Consistent Naming

Keep naming conventions consistent across the API.

Example:

- If you use `/user`, don't switch to `/customers` for the same concept in another endpoint.

5. Use Resource Identifiers for Specific Items

Append the unique identifier of a resource to its URI for operations on a single item.

Example:

- `/users/123` (specific user with ID 123)
- `/products/42` (specific product with ID 42)

```
/models
/models/231920-9-sddasda210-12-sdadasd/history
put /models/231920-9-sddasda210-12-sdadasd/history {data:"something"}
```

Get all users	GET	/users	Retrieve all users
Get a single user	GET	/users/{id}	Retrieve a user by ID
Create a user	POST	/users	Add a new user
Update a user	PUT	/users/{id}	Update details of a specific user
Delete a user	DELETE	/users/{id}	Remove a user by ID

Resource Representation

```
{
  "id": 1,
  "name": "Alice",
  "email": "alice@example.com",
  "created_at": "2023-12-01T12:34:56Z"
}
```

```
history = [
  {
    "order_id": 101,
    "product": "Laptop",
    "price": 1200.00,
```



```
order_id product price status
101 laptop 1200.00 shipped
```

Additional Guidelines for Naming

1. Avoid File Extensions

Don't include `.json` or `.xml` in URIs. The client specifies the format via HTTP headers (e.g., `Accept: application/json`).

2. Keep URIs Short and Descriptive

Use meaningful, human-readable names for resources.

Example:

-  `/users/1/orders`
-  `/usr/1/odrs`

3. Use Query Parameters for Filtering

For operations that don't fit neatly into RESTful resource structure (e.g., searching or filtering), use query parameters.

Example:

- `/products?category=electronics&price_min=100&price_max=1000`

Common Mistakes to Avoid

1. Including Actions in URIs

- Avoid: `/getUsers`, `/createOrder`.
- Use: `/users` (with `GET`) and `/orders` (with `POST`).

2. Not Representing Hierarchies Properly

- Avoid: `/orders/{orderId}/user`.
- Use: `/users/{userId}/orders/{orderId}`.

3. Using IDs Inconsistently

- Avoid: `/users/{userId}/orders/order42`.
- Use: `/users/{userId}/orders/{orderId}`.

CRUD

- Create
- Retrieve
- Update
- Delete